Software Engineering
Malika Satayeva
satayeva@kth.se

# 1 Introduction

My research direction broadly sits within explainable AI (xAI). Currently, I am working on a particular field known as Data Valuation (DV), which is the aspect I will look at in this essay. The objective of DV is to give a relative value to each individual data point in the training set. Questions regarding data quality are without a doubt important in the areas of machine learning and AI. Data is available in large quantities from a variety of sources, and its usefulness for a particular downstream task varies and is often questionable, not to mention the different regulatory restrictions there are. The issue, however, is this: how can we assign a value to a data point when value is not an intrinsic feature of the data but rather depends on the purpose for which we want to use the data?

In short, one way to divide methods in DV is based on whether they are model-agnostic or model-dependent. Data Shapley [2] is considered model-dependent in the sense I'm using here because we have to train the model on the data anyway (even though it can be used with any model), and its exact calculation involves exponentially many retrainings of the model. On one hand, we could try to estimate Data Shapley more efficiently, for instance by making use of structural assumptions that lessen the computational load. On the other hand, we might ask whether it is possible to avoid Shapley-based and other model-dependent methods altogether and instead define data value in a model-agnostic manner, for example by means of statistical distances between distributions, without relying on any particular model. I am interested in both aspects of this issue. Thus, the general question becomes: how should we assign value to data, and what properties should such a valuation have?

# 2 Lecture principles

*Not enough knowledge early; the devil is in details; hence, iterate* I think this reflects how science is generally done; I agree that the devil is in the details. I see this clearly in my research between stages of literature review & theory development and actual experiments with the data. Some assumptions which sound weak/strong in the beginning when we develop the methodology, e.g., access to a clean reference set for DV, do not happen to be so as we start experiments. The more insights we get from proven theory, synthetic and real-world experiments, the more likely we are able to spot the real limitations, not only the ones listed by the authors of existing works, in the methods (including ours) and approach them accordingly.

*Verification versus validation* I think in most of the fields, there are so called classical, or go-to, approaches. For example, in DV it is Shapley values. There is a well-defined theory behind them, we know which axioms they satisfy and that makes them appealing. But another question is: do we really need to satisfy these axioms? Speaking in the language of the lecture, are we building the right product? As I wrote in introduction, there is a particular line of research that focuses on estimating Shapley values efficiently, or finding assumptions under which they could be computed exactly in polynomial time. This implicitly assumes that Shapley values are the right notion of value for the problem at hand. Note this is not my critique of Shapley values. It is just an example of a case where the validation part is often taken for granted from previous work.

## 3 Guest lecture principles

*Understand the problem before building the solution* I also agree with the closely related statement from another lecture in this course that *understanding the problem is (more than half) of the problem.* I think this is relevant to any field. A useful distinction is between the problem space, which is about why something is needed and what it should achieve, and the solution space, which is about how to build it. In data valuation, this means that before choosing a valuation method, we first need to be clear about what *valuable* means in the particular setting. The goal could be to detect corrupted data, improve model performance, decide which points should be inspected, or something else. Even the way we use the ranking can change depending on the goal, for example whether we are interested in the top-$k$ or bottom-$k$ points. So the method only really makes sense once the problem and the intended use are clear.

*Human aspects matter in software engineering* Software is built and used by people, so their behavior affects the whole process. Behavioral Software Engineering focuses on these human and social aspects. Again, this directly connects to explainable AI and data valuation. A valuation method can score a data point or feature, but someone still has to interpret that score. In practice, the important question is what a person can actually do with that information: inspect the data, correct it, remove it, or decide that no action is needed. So it is not enough for the method to be mathematically sound, the output also has to make sense to the person using it.

## 4 Data scientists, software engineers, AI engineers

The main distinction in the chapters is between a model-centric and a system-wide view. Data science is mostly on the model-centric side, the book also puts ML engineering and MLOps there, since they are still mainly concerned with getting the model from training to deployment. Software engineering takes a broader view of the whole system, where things like maintainability, behavior in real use, and interaction with other components also matter. I think this distinction makes sense, but I do not see the roles themselves as completely separate. Both data scientists and software engineers have to take the goals and constraints of the system into account, although they may interpret them differently depending on their role. Data leakage is a good example (discussed later in Section 5.2): one role may mainly see it as a data-splitting problem, while another may think about it in terms of scenario coverage.

The system-wide view also matters for data valuation. Much of DV is naturally model-centric: value is often defined through how much a point improves a model or some validation metric. But improving that metric does not necessarily mean improving what the whole system is actually supposed to achieve. In the medical imaging project (Sections 5.1 – 5.2), an image might be valuable because annotating it would slightly improve validation accuracy, but if it takes a radiologist much longer to annotate than other images, it may not be the most useful choice for the system as a whole. The real objective is not only model performance, but also how the limited annotation time is used. This is close to the verification versus validation distinction from Section 2: we may verify that the valuation method optimizes the quantity we defined, while still asking whether that quantity is the right one. The model-agnostic direction does not automatically solve this either. Replacing the model objective with a distributional one still leaves the question of whether that objective reflects the actual system goal.

In the current state, a data scientist can build "working" software with coding assistants, and a software engineer can produce a training script that "runs". The real question in both cases is not whether the artifact executes, but whether we can tell that the whole system does what it is supposed to do. At the same time, these tools can probably shorten the path to becoming competent in areas outside one's main expertise. I think the roles will stay different, but there will probably be more overlap, and each side will need to understand the other a bit

better.

Finally, I see AI Engineering more broadly than just pipelines, deployment, and MLOps. If it stops at the model API, it misses many of the problems that appear when the ML component interacts with the rest of the system and with the people using it. The two papers discussed later in this essay show it: one brings software engineering ideas into the data pipeline, while the other shows that some problems are also organizational and depend on how different roles understand and handle them.

# 5 Paper analysis

## 5.1 "Generating and Verifying Synthetic Datasets with Requirements Engineering" [3], CAIN 2025

*Core ideas*

Vonderhaar et al. [3] treat a synthetic dataset as a software product and apply requirements engineering to it, on the premise that training data is the primary means of shaping the behavior of AI models. Requirements describe the intended behavior of the downstream model; they are then transformed into prompts for FLUX.1-schnell from HuggingFace, and each generated image is manually verified both in relation to the prompt and passed through a pre-trained YOLOv8 detector that serves as a stand-in for the downstream model. Note that it is assumed that if YOLOv8 detects the object, a model trained on that image will learn the same behavior. A data card is provided with the dataset and records which specifications it was found to meet. This gives some traceability between the resulting dataset and the original requirements.

However, using YOLOv8 as a proxy is still arguable. The paper does not test whether passing the YOLOv8 check actually means that a future model will learn the intended behavior. For example, some realistic toaster images were rejected simply because YOLOv8 confused them with similar classes.

*Relation to my research*

The paper is connected to my work because it also focuses on the training data rather than only on the model. In data valuation, the aim is to understand which data points are useful, redundant, or possibly harmful. The paper asks a related but different question: does a generated point satisfy the requirement it was created for? This is not the same as asking whether that point is useful relative to the rest of the dataset. A point may satisfy its requirement but still add very little if similar data is already present. The connection to requirements is also relevant to my setting. I assess data value relative to a target represented by a reference distribution, and requirements could make this target clearer by specifying which cases or properties are important.

*Integration into a larger AI-intensive project*

I consider a medical imaging system where new images are collected at a hospital. Before they are used, they are de-identified and checked for basic data-quality problems. They then go into an annotation queue for radiologists. Since expert annotation is expensive, data valuation could be used to rank the images so that the most useful ones are annotated first. The labelled images then enter the training pipeline, and the updated model is evaluated on a separate reference set before deployment. After deployment, the model and the incoming data can be monitored to decide when new annotation or retraining is needed.

The ideas of Vonderhaar et al. [3] would mainly affect the data-generation and data-documentation parts of this system. Requirements could specify which clinically relevant cases should be represented in the training data. If some cases are rare, synthetic images could be generated to improve their coverage and checked before being added to the pipeline. The data card could also record why the synthetic data was generated and which requirements it is supposed to

satisfy.

Data valuation would fit at the annotation and data-selection stage. Instead of sending newly collected images to radiologists in an arbitrary order, the method could prioritize the ones expected to contribute most to the training set. It could also help compare synthetic candidates and avoid adding many examples that satisfy a requirement but add little new information.

*Adaptation*
One possible extension of my work would be to connect data valuation more directly to requirements. Instead of assigning value only relative to one overall reference distribution, I could ask how much a point contributes to satisfying particular coverage requirements. One could even think of a point as having several values, one for each requirement, rather than only a single scalar value. This would make the meaning of *valuable* more explicitly dependent on the intended use of the data.

### 5.2 "Data Leakage in Automotive Perception: Practitioners' Insights" [1], CAIN 2026

*Core ideas*
Babu et al. [1] study how data leakage is understood and handled in automotive ML development through interviews with ten practitioners from design, implementation, and verification roles. One of the main findings is that leakage is not understood in exactly the same way across these roles. ML-facing roles tend to describe it as a data-splitting or validation problem, while design and verification roles think more in terms of representativeness and scenario coverage. I think the main SE message is that data leakage is not only a technical problem. The authors describe it as a socio-technical coordination problem: different roles have different information and responsibilities, so preventing leakage also requires shared definitions, traceability, and communication across the development process. The paper also shows that leakage is not limited to exact duplicates. Two images can be different at the pixel level while still correspond to essentially the same situation, for example neighboring frames from the same drive or images from the same location. Practitioners therefore use image similarity, embeddings, metadata, time, and location when reasoning about leakage. I would also be careful with the conclusions about differences between roles, since four of the ten participants from the design and verification groups had never encountered data leakage directly in their own work.

*Relation to my research*
The paper is connected with my work since it demonstrates that it is not straightforward to decide whether two data points contain essentially the same information. Two images, for example, can be very different when looked at on a pixel level and yet may still represent the same underlying situation. The point is also applicable to data valuation: if a number of points contain almost the same information, their contribution in relation to the rest of the dataset may be small even though each individual point is valid.

*Integration into a larger AI-intensive project*
I use the same medical imaging system here, but this paper affects a different part of the pipeline. A hospital can have several images from the same patient, examination, etc. If related images appear across training and validation sets, evaluation may become overly optimistic even when the images are not exact duplicates. The data-management pipeline should therefore keep patient and examination information when creating the splits and maintain an independent reference set. Similarity checks could provide an additional safeguard before newly annotated data is added. This is particularly important because the annotation queue, training pipeline, and evaluation pipeline are connected: a mistake in data handling early in the system can later appear as misleading model performance. Data valuation would fit into this system after

these safeguards. It could rank newly collected images for radiologist annotation, so that the limited annotation budget is spent first on examples that are expected to add the most useful information.

*Adaptation*
One direction would be to study how model-agnostic data valuation behaves when its reference set is not as clean as assumed. For example, I could introduce near-duplicates or samples from the same patient into the reference set and study how much the resulting ranking changes.

## 6 Evidence, originality, and GenAI-use transparency

I found both papers by browsing recent CAIN full papers, initially looking for work on explainable AI, and selected two that take a data-centric view based on their relevance to the course themes and to my own research.

I used GenAI for feedback on the draft I had written and for further brainstorming when needed, and for help with phrasing and grammar. The analysis and the claims are my own, and I am responsible for them.

## References

[1] Md Abu Ahammed Babu, Sushant Kumar Pandey, Darko Durisic, Andras Balint, and Miroslaw Staron. Data leakage in automotive perception: Practitioners' insights. In *CAIN*, 2026.

[2] Amirata Ghorbani and James Y. Zou. Data shapley: Equitable valuation of data for machine learning. In *Proceedings of the 36th International Conference on Machine Learning, ICML*, 2019.

[3] Lynn Vonderhaar, Timothy Elvira, and Omar Ochoa. Generating and verifying synthetic datasets with requirements engineering. In *CAIN*, 2025.